AI ASSISTANT CODING - ASSIGNMENT-14.4

Name: Valaboju Harshavardhanu

Roll No: 2303A52354 **Batch:** 41

Task 1: Responsive Homepage Layout for a Startup Website

Prompt:

create a webpage structure using HTML that includes a header with navigation links ,a main content section, a footer with basic information and generate responsive CSS using media queries and ensure that layout adjusts smoothly across different screen sizes, the header, content, and footer are clearly separated layout adapts correctly to mobile, tablet, and desktop views and code is clean and well-structured

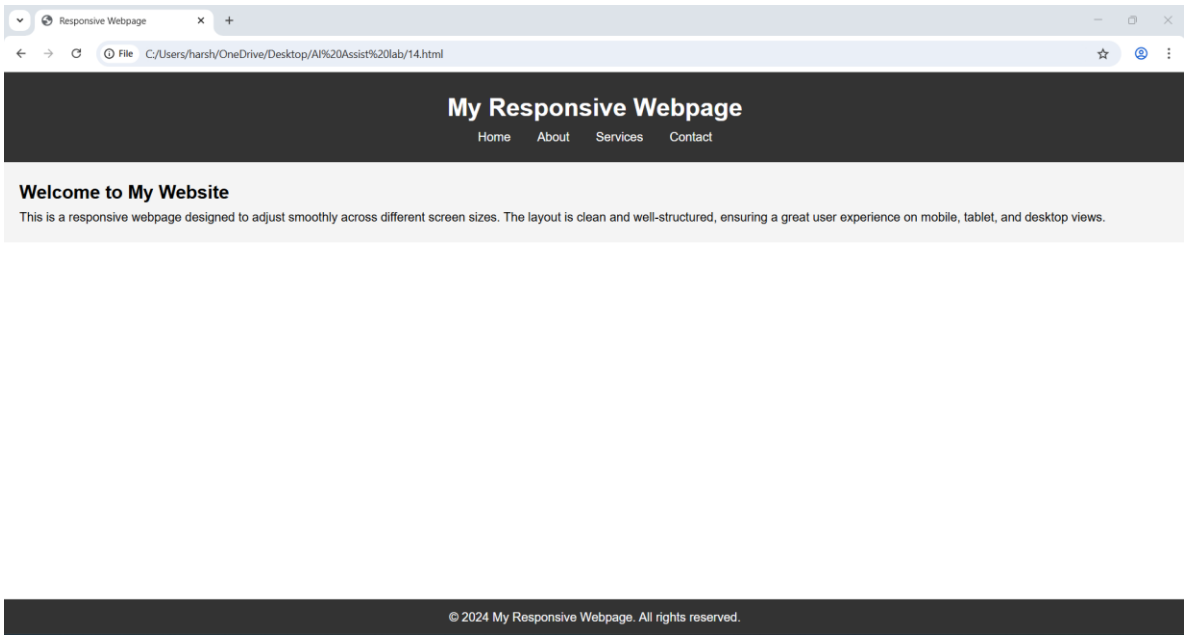
Code:

```html
<!-- create a webpage structure using HTML that includes a header with navigation
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Webpage</title>
    <style>
        /* Basic Reset */
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
        }

        /* Header */
        header {
            background-color: #333;
            color: #fff;
            padding: 20px 0;
            text-align: center;
        }

        nav a {
            color: #fff;
            margin: 0 15px;
            text-decoration: none;
        }

        /* Main Content */
        main {
            padding: 20px;
```

```
        /* Main Content */
        main {
            padding: 20px;
            background-color: #f4f4f4;
        }

        /* Footer */
        footer {
            background-color: #333;
            color: #fff;
            text-align: center;
            padding: 10px 0;
            position: fixed;
            width: 100%;
            bottom: 0;
        }

        /* Responsive Design */
        @media (max-width: 768px) {
            nav a {
                display: block;
                margin: 10px 0;
            }
        }

        @media (max-width: 480px) {
            header, main, footer {
                padding: 10px;
                text-align: center;
            }
        }
    </style>
</head>
<body>
```

```
    <header>
        <h1>My Responsive Webpage</h1>
        <nav>
            <a href="#home">Home</a>
            <a href="#about">About</a>
            <a href="#services">Services</a>
            <a href="#contact">Contact</a>
        </nav>
    </header>

    <main>
        <h2>Welcome to My Website</h2>
        <p>This is a responsive webpage designed to adjust smoothly acro
    </main>

    <footer>
        <p>&copy; 2024 My Responsive Webpage. All rights reserved.</p>
    </footer>
</body>
</html>
```

Output:



Justification:

In this task, a appropriate homepage layout was created using HTML and CSS. With the help of AI, responsive CSS using media queries was generated so that the layout adjusts properly for desktop, tablet, and mobile devices.

Task 2: Interactive Call-to-Action Button

Prompt:

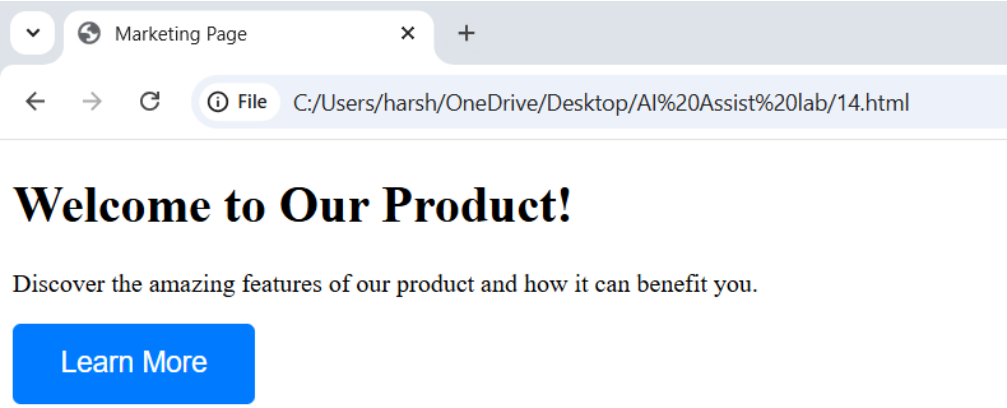
A marketing page requires a call-to-action button that responds when users interact with it, add a button to a web page using HTML and generate JavaScript that responds to the button click by displaying a message or alert and add the meaningful comments explaining the JavaScript logic

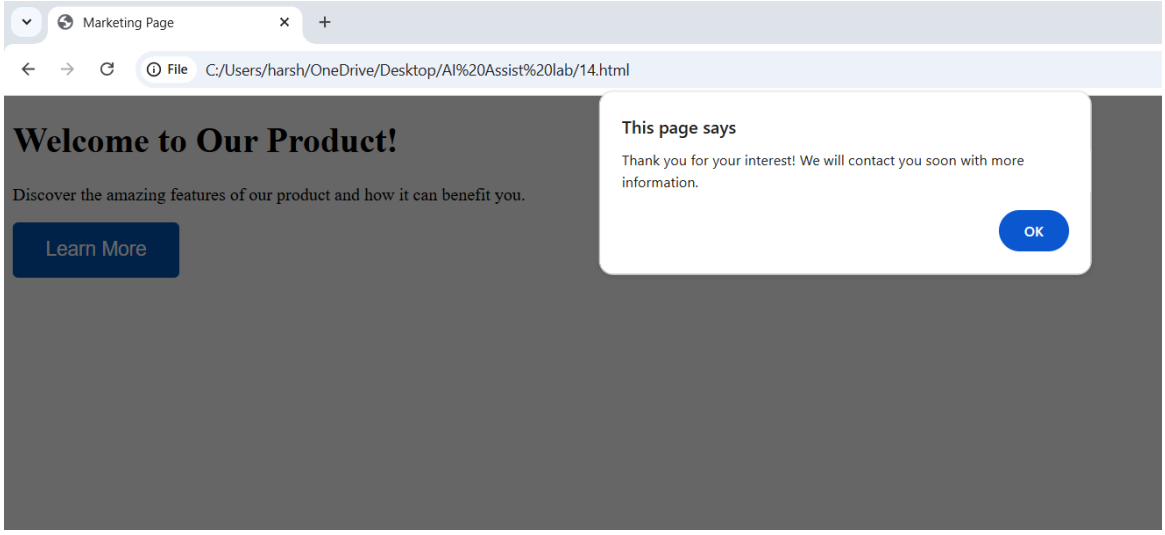
Code:

```html
<!-- A marketing page requires a call-to-action button that responds when us
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Marketing Page</title>
    <style>
        /* Basic styling for the button */
        .cta-button {
            padding: 15px 30px;
            font-size: 18px;
            color: white;
            background-color: #007BFF;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        .cta-button:hover {
            background-color: #0056b3;
        }
    </style>
```

```html
</head>
<body>
    <h1>Welcome to Our Product!</h1>
    <p>Discover the amazing features of our product and how it can benefit you.</p>
    <!-- Call-to-action button -->
    <button class="cta-button" id="ctaButton">Learn More</button>

    <script>
        // Get the button element by its ID
        const ctaButton = document.getElementById('ctaButton');

        // Add an event listener to the button that listens for 'click' events
        ctaButton.addEventListener('click', function() {
            // When the button is clicked, display an alert message to the user
            alert('Thank you for your interest! We will contact you soon with more information.');
        });
    </script>
</body>
</html>
```

Output:





Justification:

In this task, a call-to-action button was added to the page using HTML and JavaScript. AI assistance helped to generate JavaScript that displays an alert message when the button is clicked without reloading the page.

Task 3: Contact Form with Client-Side Validation

Prompt:

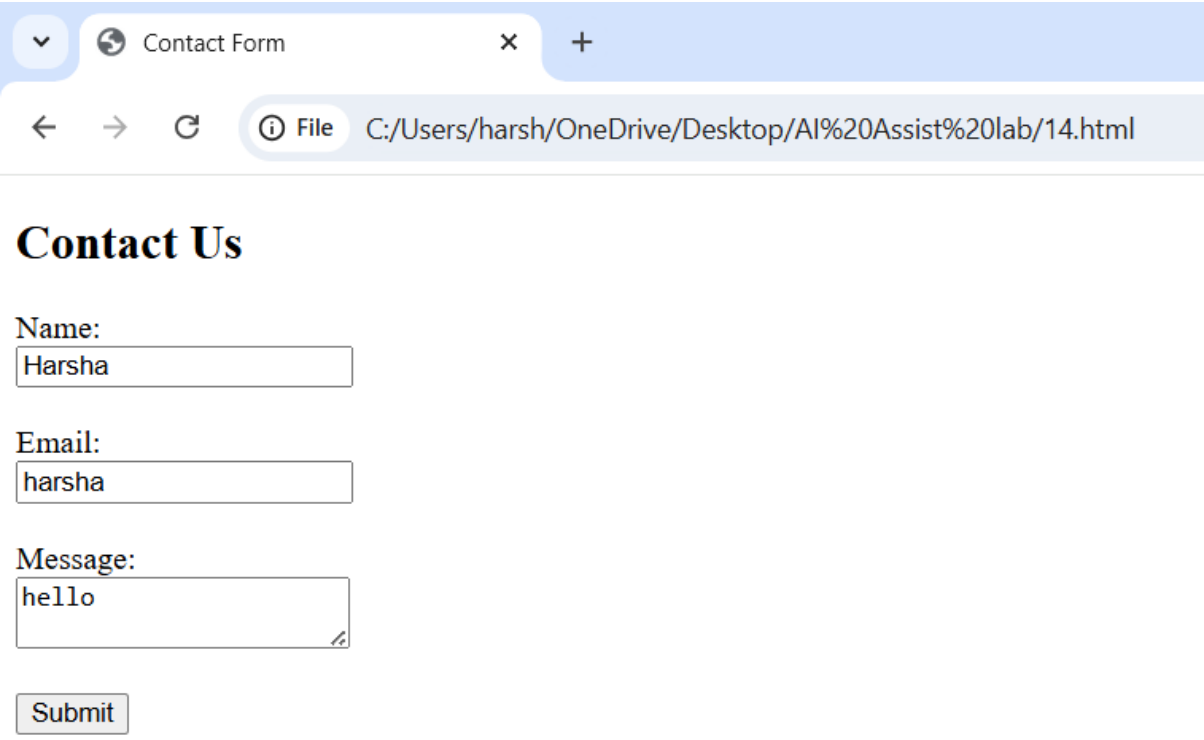
generate a contact form for a company website to collect user queries safely and correctly and design a form with fields for name, email, message and generate JavaScript validation logic that prevents empty submissions and checks for a valid email format and display clear error messages for valid input

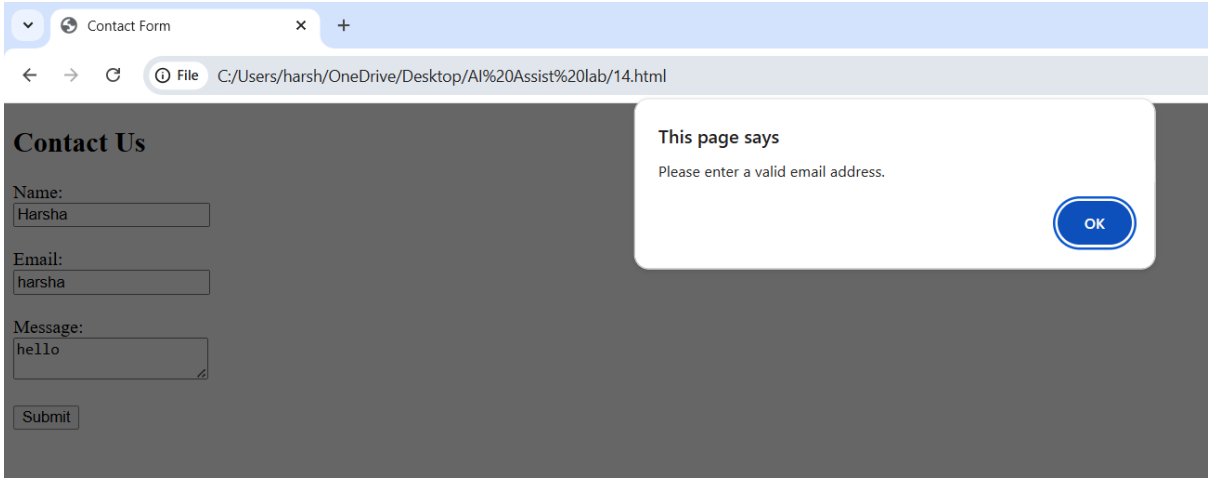
Code:

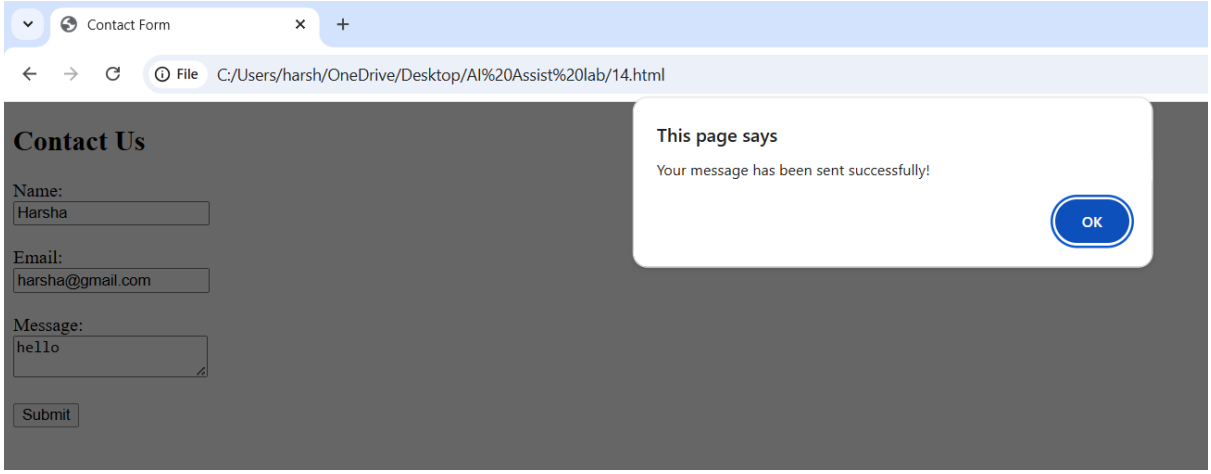
```
<!-- generate a contact form for a company website to collect user queries safely and corre
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contact Form</title>
</head>
<body>

</body>
</html>
<h2>Contact Us</h2>
<form id="contactForm">
    <label for="name">Name:</label><br>
    <input type="text" id="name" name="name"><br><br>

    <label for="email">Email:</label><br>
    <input type="text" id="email" name="email"><br><br>

    <label for="message">Message:</label><br>
    <textarea id="message" name="message"></textarea><br><br>

    <input type="submit" value="Submit">
</form>
<script>
    document.getElementById('contactForm').addEventListener('submit', function(event) {
        event.preventDefault(); // Prevent form from submitting

        // Get form values
        const name = document.getElementById('name').value.trim();
        const email = document.getElementById('email').value.trim();
        const message = document.getElementById('message').value.trim();

        // Validate form fields
        if (name === '') {
            alert('Please enter your name.');
```

```
    document.getElementById('contactForm').addEventListener('submit', function(event) {
        event.preventDefault(); // Prevent form from submitting

        // Get form values
        const name = document.getElementById('name').value.trim();
        const email = document.getElementById('email').value.trim();
        const message = document.getElementById('message').value.trim();

        // Validate form fields
        if (name === '') {
            alert('Please enter your name.');
            return;
        }

        if (email === '') {
            alert('Please enter your email.');
            return;
        }

        // Simple email format validation
        const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email)) {
            alert('Please enter a valid email address.');
            return;
        }

        if (message === '') {
            alert('Please enter your message.');
            return;
        }

        // If all validations pass, you can proceed to submit the form or display a success message
        alert('Your message has been sent successfully!');
    });
</script>
```

Output:







Justification:

In this task, a contact form with fields for name, email, and message was created. JavaScript validation generated with AI ensures that when the user enters the empty inputs those are prevented and displays the message and email format is checked.

Task 4: Dynamic List Management for a Product Page

Prompt:

A product listing page needs to add or remove items dynamically without refreshing the page, create a simple HTML list to display items and generate

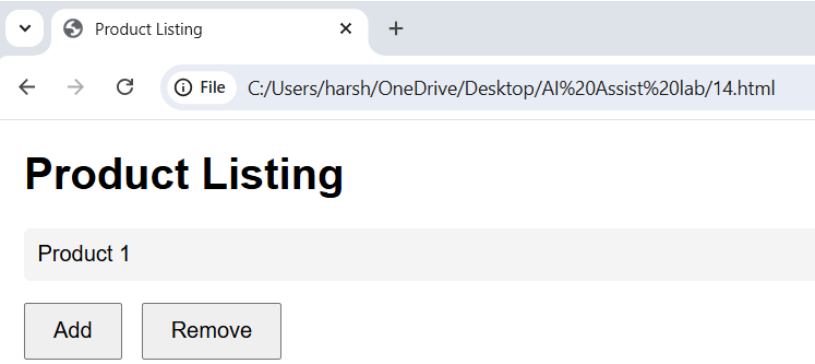
JavaScript that Adds a new item when an "Add" button is clicked, Removes the last item when a "Remove" button is clicked and ensure the DOM updates dynamically

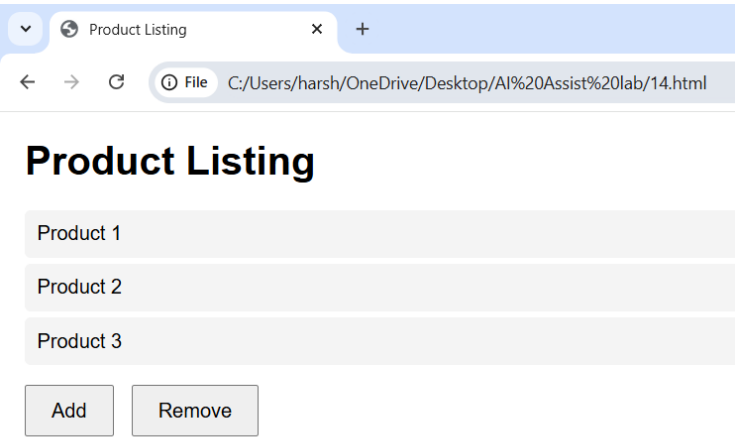
Code:

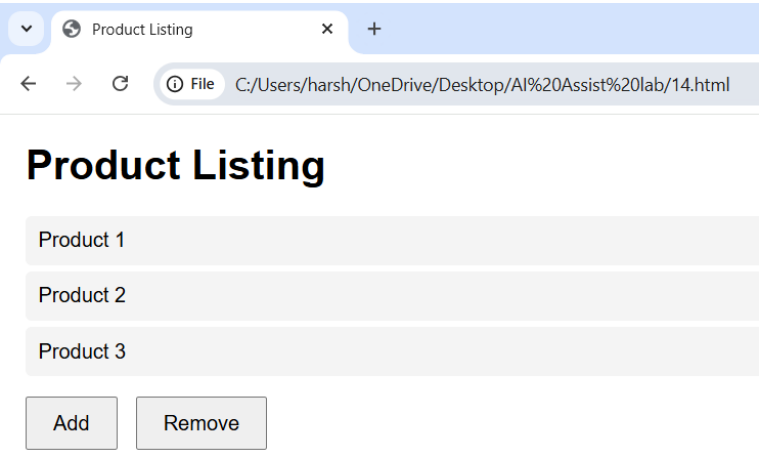
```html
<!-- A product listing page needs to add or remove items dynamically without
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Listing</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        #product-list {
            list-style-type: none;
            padding: 0;
        }
        #product-list li {
            background-color: #f4f4f4;
            margin: 5px 0;
            padding: 10px;
            border-radius: 5px;
        }
        button {
            margin-right: 10px;
            padding: 10px 20px;
            font-size: 16px;
        }
    </style>
</head>
<body>
    <h1>Product Listing</h1>
    <ul id="product-list">
        <li>Product 1</li>
        <li>Product 2</li>
        <li>Product 3</li>
    </ul>
    <button id="add-button">Add</button>
```

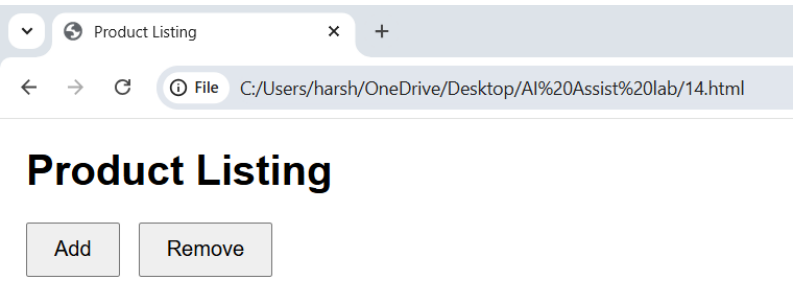
```html
    <button id="add-button">Add</button>
    <button id="remove-button">Remove</button>

    <script>
        const productList = document.getElementById('product-list');
        const addButton = document.getElementById('add-button');
        const removeButton = document.getElementById('remove-button');

        addButton.addEventListener('click', () => {
            const newItem = document.createElement('li');
            newItem.textContent = `Product ${productList.children.length + 1}`;
            productList.appendChild(newItem);
        });

        removeButton.addEventListener('click', () => {
            if (productList.children.length > 0) {
                productList.removeChild(productList.lastElementChild);
            }
        });
    </script>
</body>
</html>
```

Output:


Product Listing
File
C:/Users/harsh/OneDrive/Desktop/AI%20Assist%20lab/14.html
Product Listing
Product 1
Add
Remove


Product Listing
File
C:/Users/harsh/OneDrive/Desktop/AI%20Assist%20lab/14.html
Product Listing
Product 1
Product 2
Product 3
Add
Remove


Product Listing
File
C:/Users/harsh/OneDrive/Desktop/AI%20Assist%20lab/14.html
Product Listing
Product 1
Product 2
Product 3
Add
Remove


Product Listing
File
C:/Users/harsh/OneDrive/Desktop/AI%20Assist%20lab/14.html
Product Listing
Add
Remove

Justification:

In this task, a dynamic list was created using HTML and JavaScript to manage items on a product page. With the help of AI assistance, JavaScript logic was generated to add new items and remove existing ones using buttons. The list updates instantly on the web page without refreshing. This task helped in understanding DOM manipulation and dynamic content updates.

Task 5: Modal Popup for User Notifications

Prompt:

A user notification popup for announcements or alerts on a website, and generate HTML structure for a model popup and CSS styling with a semi-transparent background overlay and JavaScript to open and close the modal and Include multiple ways to close the modal (button or overlay click)

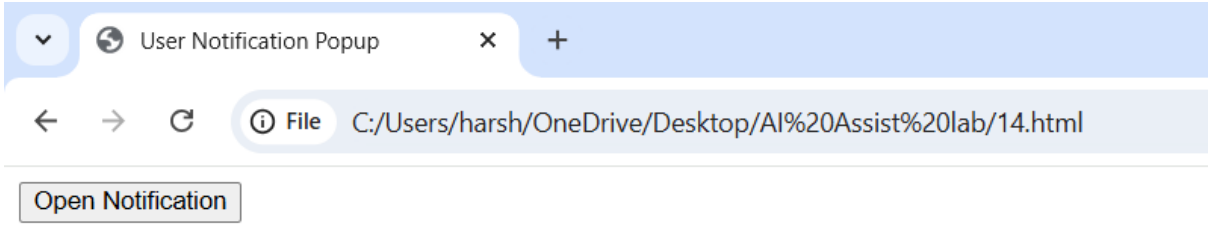
Code:

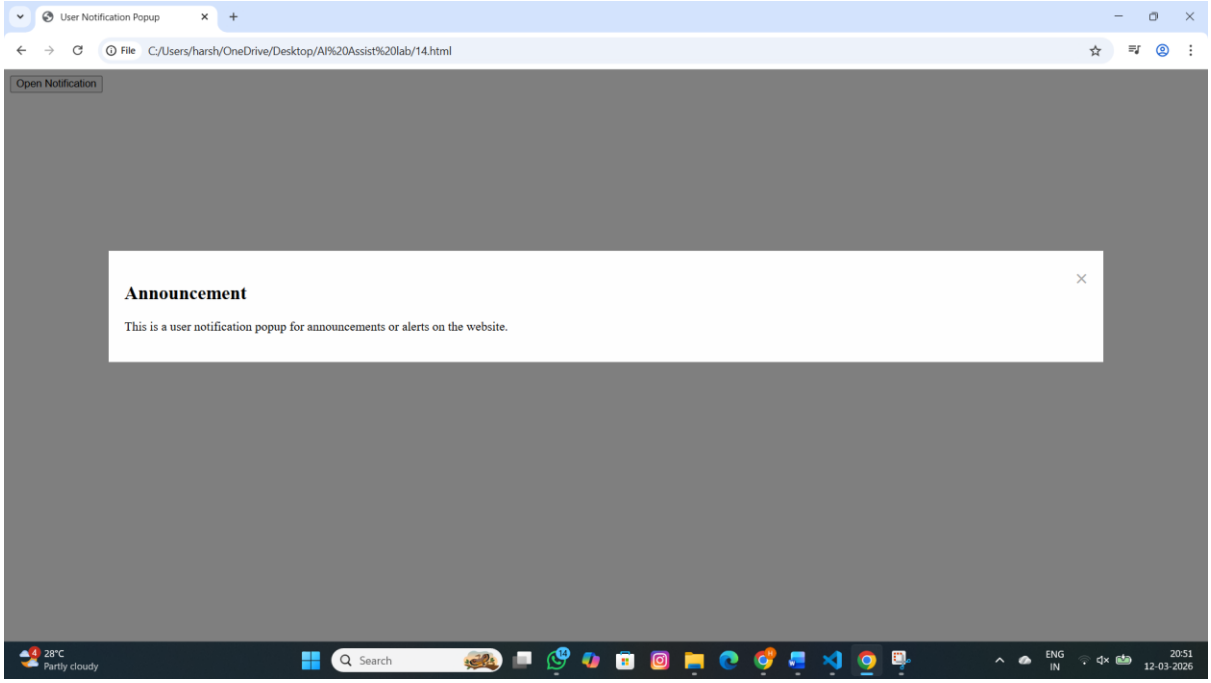
```
<!-- A user notification popup for announcements or alerts on a website, and generate HTM
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>User Notification Popup</title>
    <style>
        /* Modal container */
        .modal {
            display: none; /* Hidden by default */
            position: fixed; /* Stay in place */
            z-index: 1; /* Sit on top */
            left: 0;
            top: 0;
            width: 100%; /* Full width */
            height: 100%; /* Full height */
            overflow: auto; /* Enable scroll if needed */
            background-color: rgba(0, 0, 0, 0.5); /* Semi-transparent background */
        }

        /* Modal content */
        .modal-content {
            background-color: #fefefe;
            margin: 15% auto; /* Center the modal */
            padding: 20px;
            border: 1px solid #888;
            width: 80%; /* Could be more or less, depending on screen size */
        }

        /* Close button */
        .close {
            color: #aaa;
            float: right;
            font-size: 28px;
            font-weight: bold;
        }
```

```html
        .close:hover,
        .close:focus {
            color: black;
            text-decoration: none;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <!-- Trigger/Open The Modal -->
    <button id="openModalBtn">Open Notification</button>

    <!-- The Modal -->
    <div id="myModal" class="modal">
        <!-- Modal content -->
        <div class="modal-content">
            <span class="close">&times;</span>
            <h2>Announcement</h2>
            <p>This is a user notification popup for announcements or alerts on the website.</p>
        </div>
    </div>

    <script>
        // Get the modal
        var modal = document.getElementById("myModal");

        // Get the button that opens the modal
        var btn = document.getElementById("openModalBtn");

        // Get the <span> element that closes the modal
        var span = document.getElementsByClassName("close")[0];
```

```html
        // When the user clicks the button, open the modal
        btn.onclick = function() {
            modal.style.display = "block";
        }

        // When the user clicks on <span> (x), close the modal
        span.onclick = function() {
            modal.style.display = "none";
        }

        // When the user clicks anywhere outside of the modal, close it
        window.onclick = function(event) {
            if (event.target == modal) {
                modal.style.display = "none";
            }
        }
    </script>
</body>
</html>
```

Output:





Justification:

In this task, a modal popup was designed to display notifications or announcements on a web page. AI assistance helped generate the HTML structure, CSS styling with an overlay, and JavaScript logic to control opening and closing of the modal. The popup can be closed using a button or by clicking outside the modal. This task helped in understanding how interactive UI components improve user experience.